

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-208БВ-25

Студент: Лунева Л.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.10.26

Москва, 2026

Постановка задачи

Вариант 13.

Цель работы: приобретение практических навыков в управлении процессами в ОС и обеспечении обмена данными между процессами посредством каналов.

Задание: составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должна создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Группа вариантов 3. Родительский процесс создаёт два дочерних процесса. Родительский и дочерние процессы должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процессы child1 и child2 производят работу над строками; child1 и child2 можно «соединить» между собой дополнительным каналом. Child2 пересылает результат своей работы родительскому процессу через pipe2. Родительский процесс выводит полученный результат в стандартный поток вывода.

Условие варианта: Child1 переводит строки в нижний регистр. Child2 превращает все пробельные символы в символ «_».

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void);` – создаёт дочерний процесс, копию вызывающего. В родителе возвращает PID ребёнка, в ребёнке — 0, при ошибке — -1.
- `int pipe(int fd[2]);` – создаёт однонаправленный канал; `fd[0]` — конец для чтения, `fd[1]` — конец для записи.
- `int dup2(int oldfd, int newfd);` – делает дескриптор `newfd` копией `oldfd`; используется для перенаправления стандартных потоков ввода и вывода дочерних процессов на концы каналов.
- `int execv(const char *path, char *const argv[]);` – заменяет образ текущего процесса программой `path`; открытые дескрипторы при этом сохраняются. Возвращает управление только при ошибке.
- `ssize_t read(int fd, void *buf, size_t count);` – читает до `count` байт из дескриптора `fd`; возвращает число прочитанных байт, 0 при конце данных (EOF) или -1 при ошибке.
- `ssize_t write(int fd, const void *buf, size_t count);` – записывает `count` байт в дескриптор `fd`; возвращает число записанных байт или -1 при ошибке.
- `int close(int fd);` – закрывает файловый дескриптор.
- `pid_t wait(int *status);` – ожидает завершения любого дочернего процесса.
- `sighandler_t signal(int signum, sighandler_t handler);` – устанавливает реакцию процесса на сигнал; используется для игнорирования сигнала SIGPIPE.
- `void _exit(int status);` – немедленно завершает процесс с указанным кодом.

Решение состоит из трёх программ: `parent` (родительский процесс), `child1` и `child2` (дочерние процессы). Для взаимодействия используются три канала: `pipe1` передаёт данные от родителя к `child1`, `pipe3` — от `child1` к `child2`, `pipe2` — от `child2` обратно к родителю.

Родительский процесс создаёт три канала и с помощью `fork` порождает два дочерних процесса. В каждом из них стандартный ввод и вывод перенаправляются вызовом `dup2` на нужные концы каналов: у `child1` `stdin` связан с `pipe1`, а `stdout` — с `pipe3`; у `child2` `stdin` связан с `pipe3`, а `stdout` — с `pipe2`. После этого дочерний процесс закрывает все исходные дескрипторы каналов и вызовом `execv` заменяет свой образ программой `child1` или `child2`. Благодаря тому, что дескрипторы сохраняются при `execv`, дочерние программы ничего не знают о каналах: они просто читают из стандартного ввода и пишут в стандартный вывод.

Программа `child1` в цикле читает данные из `stdin`, переводит латинские заглавные буквы в строчные и записывает результат в `stdout`. Программа `child2` аналогично заменяет все пробельные символы (пробел, табуляцию и др.) на символ «`_`». Символ перевода строки при этом сохраняется, чтобы не нарушать разбиение ввода на строки.

Родительский процесс закрывает неиспользуемые концы каналов и оставляет только конец записи `pipe1` и конец чтения `pipe2`. В основном цикле он читает строку от пользователя, записывает её в `pipe1`, затем читает ответ из `pipe2` и выводит его в `stdout`. Поскольку обе операции над строкой не меняют её длину, родитель читает из `pipe2` в цикле ровно столько байт, сколько отправил в `pipe1`; это позволяет корректно обрабатывать строки произвольной длины, даже если данные приходят по частям.

Ввод завершается пустой строкой или комбинацией `Ctrl+D`. После этого родитель закрывает `pipe1`; `child1` получает конец данных (`read` возвращает 0) и завершается, что, в свою очередь, закрывает `pipe3` и приводит к завершению `child2`. Родитель дожидается завершения обоих дочерних процессов с помощью `wait`. Важно, что каждый процесс закрывает все неиспользуемые концы каналов: в противном случае конец данных не был бы получен и программа зависла бы.

Результат каждого системного вызова проверяется; при ошибке в стандартный поток ошибок выводится сообщение с указанием процесса и причины, и процесс завершается с ненулевым кодом. Дополнительно родитель игнорирует сигнал `SIGPIPE`: если дочерний процесс аварийно завершился, запись в канал без читателя не убивает родителя молча, а возвращает ошибку, которая обрабатывается штатно. В соответствии с требованиями к работе, для ввода-вывода используются только системные вызовы, без библиотеки `stdio`.

Код программы

parent.c

```
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

static char PROGRAM_CHILD_1[] = "child1";
static char PROGRAM_CHILD_2[] = "child2";

static void print_error(const char *msg) {
    size_t len = 0;
    while (msg[len] != '\0')
        ++len;
    write(STDERR_FILENO, msg, len);
    _exit(EXIT_FAILURE);
}

int main(void) {
```

```

if (signal(SIGPIPE, SIG_IGN) == SIG_ERR)
    print_error("parent: failed to set SIGPIPE handler\n");

int pipe1[2];
if (pipe(pipe1) == -1)
    print_error("parent: failed to create pipe1\n");

int pipe3[2];
if (pipe(pipe3) == -1)
    print_error("parent: failed to create pipe3\n");

int pipe2[2];
if (pipe(pipe2) == -1)
    print_error("parent: failed to create pipe2\n");

const pid_t child1 = fork();
if (child1 == -1)
    print_error("parent: failed to spawn child1\n");

if (child1 == 0) {
    if (dup2(pipe1[0], STDIN_FILENO) == -1 || dup2(pipe3[1], STDOUT_FILENO) == -1)
        print_error("parent: dup2 failed for child1\n");

    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    close(pipe3[0]); close(pipe3[1]);

    char *const args[] = {PROGRAM_CHILD_1, NULL};
    execv("./child1", args);
    print_error("parent: failed to exec child1\n");
}

const pid_t child2 = fork();
if (child2 == -1)
    print_error("parent: failed to spawn child2\n");

if (child2 == 0) {
    if (dup2(pipe3[0], STDIN_FILENO) == -1 || dup2(pipe2[1], STDOUT_FILENO) == -1)
        print_error("parent: dup2 failed for child2\n");

    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    close(pipe3[0]); close(pipe3[1]);

    char *const args[] = {PROGRAM_CHILD_2, NULL};
    execv("./child2", args);
    print_error("parent: failed to exec child2\n");
}

close(pipe1[0]);
close(pipe2[1]);
close(pipe3[0]);
close(pipe3[1]);

char buf[4096];
ssize_t bytes;

while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) != 0) {
    if (bytes < 0)
        print_error("parent: failed to read from stdin\n");

    if (buf[0] == '\n')
        break;

    if (write(pipe1[1], buf, bytes) != bytes)

```

```

        print_error("parent: failed to write to child1\n");

        ssize_t total = 0;
        while (total < bytes) {
            ssize_t got = read(pipe2[0], buf + total, bytes - total);
            if (got <= 0)
                print_error("parent: failed to read from child2\n");
            total += got;
        }

        if (write(STDOUT_FILENO, buf, total) != total)
            print_error("parent: failed to write to stdout\n");
    }

    close(pipe1[1]);
    close(pipe2[0]);

    wait(NULL);
    wait(NULL);

    return 0;
}

```

child1.c

```

#include <stdlib.h>
#include <unistd.h>
#include <ctype.h>

static void print_error(const char *msg) {
    size_t len = 0;
    while (msg[len] != '\0')
        ++len;
    write(STDERR_FILENO, msg, len);
    _exit(EXIT_FAILURE);
}

int main(void) {
    char buf[4096];
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) != 0) {
        if (bytes < 0)
            print_error("child1: failed to read from stdin\n");

        for (ssize_t i = 0; i < bytes; ++i)
            buf[i] = tolower(buf[i]);

        if (write(STDOUT_FILENO, buf, bytes) != bytes)
            print_error("child1: failed to write to stdout\n");
    }

    return 0;
}

```

child2.c

```

#include <stdlib.h>
#include <unistd.h>
#include <ctype.h>

static void print_error(const char *msg) {
    size_t len = 0;
    while (msg[len] != '\0')
        ++len;
}

```

```

        write(STDERR_FILENO, msg, len);
        _exit(EXIT_FAILURE);
    }

    int main(void) {
        char buf[4096];
        ssize_t bytes;

        while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) != 0) {
            if (bytes < 0)
                print_error("child2: failed to read from stdin\n");

            for (ssize_t i = 0; i < bytes; ++i) {
                if (isspace(buf[i]) && buf[i] != '\n')
                    buf[i] = '_';
            }

            if (write(STDOUT_FILENO, buf, bytes) != bytes)
                print_error("child2: failed to write to stdout\n");
        }

        return 0;
    }

```

Makefile

```

CC = gcc
CFLAGS = -Wall -Wextra

all: parent child1 child2

parent: parent.c
    $(CC) $(CFLAGS) -o parent parent.c

child1: child1.c
    $(CC) $(CFLAGS) -o child1 child1.c

child2: child2.c
    $(CC) $(CFLAGS) -o child2 child2.c

clean:
    rm -f parent child1 child2

```

Протокол работы программы

Сборка:

```

$ make
gcc -Wall -Wextra -o parent parent.c
gcc -Wall -Wextra -o child1 child1.c
gcc -Wall -Wextra -o child2 child2.c

```

Тест 1. Интерактивный ввод строки:

```

$ ./parent
Hello WORld! I'm NOT a ChILD    OkaY?!
hello_world!_i'm_not_a_child____okay?!

```

Тест 2. Несколько строк, табуляция и несколько пробелов подряд:

```

$ printf 'First Line\nSecond    LINE\twith TAB\nMixED CaSe 123    !!!\n' | ./parent
first_line
second__line_with_tab
mixed_case_123__!!!

```

Тест 3. Обработка ошибки: исполняемый файл child1 отсутствует:

```
$ mv child1 child1.bak
$ echo "test" | ./parent
parent: failed to exec child1
parent: failed to write to child1
$ echo $?
1
```

Вывод

В ходе лабораторной работы я освоила создание процессов с помощью `fork`, замену образа процесса через `execv`, организацию обмена данными между процессами через неименованные каналы и перенаправление стандартных потоков с помощью `dup2`. Была написана программа из трёх процессов, соединённых цепочкой каналов, которая переводит строки в нижний регистр и заменяет пробельные символы на «_», с обработкой ошибок всех системных вызовов.

Основные трудности были связаны с корректным закрытием дескрипторов: незакрытый конец канала приводит к тому, что процесс никогда не получает конец данных, и программа зависает. Также важным оказался порядок вызовов `dup2` и `close` в дочернем процессе. Кроме того, при тестировании аварийного запуска выяснилось, что запись в канал без читателя приводит к молчаливому завершению родителя сигналом `SIGPIPE`, что было исправлено игнорированием этого сигнала.